

PAC Framework

Signal Generation Reference

A complete guide to synthetic iEEG generation
with phase-amplitude coupling

Version 0.10.0 | Schema version 0.10.0

Table of Contents

- 1. Overview & Architecture**
- 2. The Signal Pipeline - Step by Step**
- 3. Oscillator Populations**
 - 3.1 Frequency Drift (PAF Drift)
 - 3.2 Waveform Shape (Harmonic Injection)
 - 3.3 Burst Mode
 - 3.4 Artifacts
- 4. Background (1/f) Populations**
- 5. Line Noise Populations**
- 6. Couplings**
 - 6.1 Phase-to-Amplitude Coupling (PAC)
 - 6.2 Event-Modulated Coupling Depth
 - 6.3 Phase-to-Phase Coupling (PPC)
- 7. Population-to-Channel Projection**
- 8. Per-Channel Independent Noise**
- 9. Ground-Truth Bundle**
- 10. Experiment Design - Sessions & Events**
- 11. Using the GUI - Tab by Tab**
 - 11.1 Subject Designer Tab
 - 11.2 Signal Config Tab
 - 11.3 Data Browser Tab
- 12. Programmatic API (no GUI)**
- 13. Saving & Loading Subjects**
- 14. Deterministic Reproducibility**
- 15. Parameter Quick Reference**

1. Overview & Architecture

The PAC Framework is a Python library and PyQt6 desktop application for synthesising realistic intracranial EEG (iEEG) signals with configurable neural oscillations, 1/f background activity, line noise, and phase-amplitude coupling (PAC). Its primary purpose is to produce ground-truth-labelled synthetic datasets for benchmarking PAC analysis algorithms.

Core design principles

- Immutable data model - Session, Channels, Events, Timeline are all frozen dataclasses. No in-place mutation anywhere in the scientific layer.

- GUI-free
used from

File layout

```
pac_framework/
__init__.py      Public API - import pac_framework as pac
core/
  data_model.py  Timeline, Channels, Events, Session, Subject, Result
  seed_util.py   derive(parent, *labels) -> int (BLAKE2b)
  manifest_migrations.py Schema version migration chain (0.0.0 -> 0.10.0)
generator/
  config.py      All Pydantic frozen models (SignalConfig, etc.)
  oscillator.py  synth_oscillator - band-limited noise AM model
  bursts.py      burst_envelope - Poisson burst gating
  background.py  synth_background - 1/f spectral shaping
  line_noise.py  synth_line_noise - deterministic harmonics
  couplings.py   apply_phase_to_amplitude, build_window_envelope
  pipeline.py    apply_couplings - topological ordering + PAC apply
  projection.py  build_projection_matrix - population -> channel
  scheduling.py  schedule_class_events - Poisson event scheduler
  runner.py      build_sessions, generate_signals (public entry points)
gui/
  main_window.py MainWindow hub (toolbar, Build, Generate, Save, Load)
  subject_state.py SubjectState - GUI-only workspace state
tabs/
  subject_designer.py Tab 1 - metadata, sessions, event catalogs, shafts
  signal_config.py   Tab 2 - populations, couplings, projection
  data_browser.py    Tab 3 - channel viewer, timeline preview
widgets/
  oscillator_advanced_dialog.py Advanced oscillator params dialog
  coupling_modulation_dialog.py Event modulation dialog
  timeline_preview.py      Event raster / timeline widget
  tracked_viewer.py        Multi-channel waveform viewer
persistence.py             save_subject / load_subject (HDF5 + JSON manifest)
```

2. The Signal Pipeline - Step by Step

When you press Generate Signals (Ctrl+G) the following operations are executed sequentially for each session. All steps are pure NumPy/SciPy - no Qt is involved.

Step 1 - Seed derivation

A deterministic seed tree is built from the master seed using BLAKE2b hashing:

```
subject_seed = derive(master_seed, 'subject', subject_name)
session_seed = derive(subject_seed, 'session', session_index)
pop_seed     = derive(session_seed, 'population', population_id)
channel_seed = derive(session_seed, 'channel', channel_index)
```

Changing only the master seed regenerates every waveform independently; changing only a population ID changes only that population's seed branch.

Step 2 - Oscillator synthesis (per population)

Each OscillatorPopulation is synthesised independently by `synth_oscillator()`. The result is an OscillatorOutput(carrier, phase) pair. See Section 3 for the full sub-pipeline.

Step 3 - Chi trajectory computation

For each PhaseToAmpCoupling, a $\chi(t)$ time series is built. If no EventModulation entries exist, $\chi(t)$ is a constant array equal to `c.chi`. If event modulations are present, each event occurrence stamps a Tukey window of height (`peak_chi - baseline_chi`) into the trajectory, and multiple modulations are combined by elementwise maximum. See Section 6.2.

Step 4 - Coupling application (topological order)

`apply_couplings()` sorts populations by Kahn's topological algorithm (driver before target), then for each PAC coupling applies the von Mises envelope to the target carrier using the driver's phase. See Section 6.1.

Step 5 - Population-to-channel projection

The (`n_populations` x `n_samples`) carrier matrix `X` is left-multiplied by `M.T` to produce an (`n_channels` x `n_samples`) channel signal matrix. `M` is built by `build_projection_matrix()` from the channel layout. See Section 7.

```
channel_signals = M.T @ X # (n_channels, n_samples)
```

Step 6 - Line noise addition

All LineNoisePopulations are summed into a single trace and broadcast-added identically to every channel (apparatus-level noise).

Step 7 - Background (1/f) addition

All BackgroundPopulations are summed and broadcast-added identically across channels (shared neural background).

Step 8 - Per-channel independent Gaussian noise

Each channel receives independent Gaussian noise drawn from $N(0, \text{channel_noise_sd}^2)$ using a channel-specific seed. This models electrode-level thermal/measurement noise.

Step 9 - Ground-truth bundle assembly

All intermediate arrays (pre-coupling carriers, instantaneous phases, χ trajectories, line noise traces, background traces, projection matrix, population order, channel order, full SignalConfig JSON) are packed into `session.ground_truth` dict and stored with the session.

3. Oscillator Populations

An OscillatorPopulation is the primary neural signal source. Each population represents one neural group oscillating in a specific frequency band in a specific brain region. Multiple populations can coexist and be coupled.

Core parameters

Parameter	Type	Default	Description
id	str	(required)	Unique identifier used in couplings and ground truth.
center_frequency	float	(required)	Center frequency in Hz (must be > 0).
bandwidth	float	(required)	Bandwidth in Hz. 0 = pure sinusoid; > 0 = band-limited AM noise model.
amplitude	float	(required)	Output amplitude in uV (carrier.std() == amplitude after scaling).
region	str	(required)	Brain region label. Used by region_match projection to select channels.
seed_tag	str	'default'	Additional seed disambiguation tag (rarely needed unless duplicating configs).

The two carrier models

bandwidth = 0: Pure sinusoid

The carrier is a pure cosine at the center frequency, optionally with PAF drift applied to the instantaneous frequency:

```
carrier(t) = sqrt(2) * cos(phase(t))
phase(t)   = 2*pi * cumsum(f(t)) / sfreq
f(t)       = center_frequency (constant when sigma_hz=0)
```

With bandwidth=0 there is no RNG involved in the carrier itself (pure determinism), so every channel receiving this population gets an identical sinusoid before projection.

bandwidth > 0: Band-limited AM noise model

White Gaussian noise is low-pass filtered at cutoff = bandwidth/2, then amplitude-modulated by the cosine carrier. This produces a realistic oscillatory burst with natural amplitude variability:

```
envelope(t) = lowpass_filter(white_noise, cutoff=bandwidth/2)
carrier(t)  = envelope(t) * cos(phase(t))
```

The filter uses a 4th-order Butterworth applied with filtfilt (zero-phase). 2 seconds of throwaway samples prefix the signal to absorb filter edge transients.

3.1 Frequency Drift (PAF Drift)

The peak alpha frequency (PAF) of real neural oscillations drifts slowly over time. PAFDrift models this as an Ornstein-Uhlenbeck (OU) process on the instantaneous frequency:

```
f[n+1] = center + (f[n] - center) * exp(-dt/tau)
        + sigma * sqrt(1 - exp(-2*dt/tau)) * z[n]
where z[n] ~ N(0, 1), dt = 1/sfreq
f[n] is clipped to >= 0.1 Hz
```

Parameter	Type	Default	Description
sigma_hz	float	0.0	Standard deviation of frequency fluctuations in Hz. 0 = no drift (pure sinusoid at center_frequency).
tau_seconds	float	5.0	Correlation time of the OU process in seconds. Larger = slower, smoother drift.

Note: PAF drift is active even with bandwidth=0. You can have a pure sinusoid with a slowly drifting frequency.

3.2 Waveform Shape (Harmonic Injection)

Real neural oscillations are not perfect sinusoids. WaveformShape injects controlled harmonics to produce sharp peaks, flat troughs, or rise/decay asymmetry - hallmarks of theta, alpha and slow oscillations:

```

carrier += sharpness_harmonics: 0.3*|s|*cos(2*phase + phi) + 0.1*|s|*cos(4*phase + phi)
                                phi = 0 if s>0, pi if s<0
carrier += asymmetry_harmonics: 0.2*|a|*cos(3*phase + phi) + 0.05*|a|*cos(5*phase + phi)
                                phi = pi/2 if a>0, -pi/2 if a<0

```

Parameter	Type	Default	Description
peak_trough_sharpness	float [-1,1]	0.0	Positive = sharper peaks, flatter troughs (like hippocampal theta). Negative = flatter peaks, sharper troughs.
rise_decay_asymmetry	float [-1,1]	0.0	Positive = fast rise, slow decay. Negative = slow rise, fast decay.

3.3 Burst Mode

By default oscillators are continuous. Setting mode='bursty' gates the carrier with a stochastic on/off envelope. Bursts are Poisson-distributed, with log-normal duration and raised-cosine tapering to avoid hard edges:

Parameter	Type	Default	Description
mode	str	'continuous'	'continuous' or 'bursty'. Bursty gates the carrier with a stochastic envelope.
rate_hz	float	0.5	Mean burst rate in Hz (bursts per second). Active only when mode='bursty'.
duration_cycles_mean	float	3.0	Mean burst duration in carrier cycles.
duration_cycles_sd	float	0.5	Standard deviation of burst duration. 0 = fixed length.
refractory_cycles	float	1.0	Minimum gap between bursts in carrier cycles (refractory period).

Note: Burst mode affects the entire recording - there is no window-alignment to events. To make bursting correlate with task events, combine burst mode with event-modulated coupling depth (Section 6.2) on a gamma population.

3.4 Artifacts

ArtifactConfig injects sharp-edge transient spikes into the carrier after amplitude scaling. This models electrode movement artifacts, saturation clipping, and similar contamination:

Parameter	Type	Default	Description
rate_hz	float	0.0	Artifact injection rate in Hz. 0 = no artifacts.

amplitude_mult

float

3.0

Spike amplitude as a multiple of the carrier's std. Default 3x gives salient but realistic artifacts.

width_samples	int	5	Width of each transient spike in samples. At 1000 Hz, 5 samples = 5 ms.
---------------	-----	---	---

Note: Artifact rate_hz=0 (default) produces no artifacts. Spikes are randomly positive or negative.

4. Background (1/f) Populations

Real iEEG signals contain aperiodic 1/f like background power that is not attributable to any specific oscillation. BackgroundPopulation models this via spectral shaping of white noise in the frequency domain:

$$A(f) = 1 / \sqrt{f^{\text{slope}} + \text{knee_hz}^{\text{slope}}} \quad (\text{knee_hz} > 0, \text{ Lorentzian form})$$

$$A(f) = 1 / f^{(\text{slope}/2)} \quad (\text{knee_hz} = 0, \text{ pure power law})$$

DC component ($f=0$) is pinned to $f=1$ to avoid singularity.

The output is normalised so $\text{std} == \text{amplitude}$ (uV). Background is shared across all channels (broadcast) - it models a common neural background, not independent electrode noise.

Parameter	Type	Default	Description
id	str	(required)	Unique identifier for this background population.
slope	float	1.5	Spectral exponent. Typical iEEG: 1.0-2.5. Higher = more low-frequency dominated.
knee_hz	float	0.0	Spectral knee in Hz. Above the knee the slope flattens. 0 = pure power law.
amplitude	float	1.0	Output amplitude in uV (std of the trace).
seed_tag	str	'default'	Seed disambiguation tag.

5. Line Noise Populations

LineNoisePopulation produces a deterministic sum of sinusoids at a fundamental frequency and its harmonics, modelling electrical interference from power mains:

$$\text{trace}(t) = \sum_k \text{amplitude_per_harmonic}[k] * \sqrt{2} * \cos(2 * \pi * \text{harmonics}[k] * \text{frequency} * t)$$

The $\sqrt{2}$ factor makes each harmonic's RMS = $\text{amplitude_per_harmonic}[k]$.

Line noise is deterministic (no RNG) and is broadcast identically to all channels since mains interference is apparatus-level.

Parameter	Type	Default	Description
id	str	(required)	Unique identifier.
frequency	float	60.0	Fundamental frequency in Hz. 60 Hz (US/Japan) or 50 Hz (Europe).
harmonics	list[int]	[1, 2, 3]	List of harmonic indices. [1,2,3] = fundamental + 2nd + 3rd harmonic.
amplitude_per_harmonic	list[float]	[1.0,0.3,0.1]	RMS amplitude in uV for each harmonic in the harmonics list.

6. Couplings

Couplings define directional relationships between populations. They are validated against the population graph (no unknown IDs, no cycles) before synthesis. Multiple couplings can share drivers or targets.

6.1 Phase-to-Amplitude Coupling (PAC)

PAC is the core feature of the framework. The amplitude of a high-frequency population (the target) is modulated by the phase of a low-frequency population (the driver). This is the canonical theta-gamma PAC mechanism.

The modulation uses the von Mises kernel:

$$E(\phi; \chi, \phi_0, \kappa) = (1 - \chi) + \chi * \exp(\kappa * \cos(\phi - \phi_0) - \kappa)$$

At $\phi = \phi_0$: $E = 1.0$ (maximum - carrier amplitude unchanged)

At $\phi = \phi_0 + \pi$: $E = (1 - \chi) + \chi * \exp(-2 * \kappa)$ (minimum)

$\chi = 0$: $E = 1.0$ everywhere (no modulation)

$\kappa = 0$: $E = 1.0$ everywhere (no directional preference)

The modulated carrier is:

$$\text{target_carrier_out}(t) = \text{target_carrier}(t) * E(\text{driver_phase}(t))$$

Parameter	Type	Default	Description
driver	str	(required)	Population ID of the phase driver (low-frequency oscillation).
target	str	(required)	Population ID of the amplitude target (high-frequency oscillation).
chi	float [0,1]	0.5	Baseline coupling depth. 0 = no coupling, 1 = full coupling.
phi_0	float (rad)	0.0	Preferred coupling phase in radians. GUI shows degrees. $\phi_0=0$ = coupling peaks at cosine peak of driver.
kappa	float ≥ 0	2.0	Concentration parameter of the von Mises distribution. Higher = narrower coupling window.

Note: The driver's carrier amplitude is NOT modified - only the target is affected. The driver's instantaneous phase is always taken from the original synthesised phase (pre-coupling), regardless of any coupling chain.

6.2 Event-Modulated Coupling Depth

G5b allows the coupling depth $\chi(t)$ to vary over time, rising transiently after experimental events. This models task-locked PAC enhancement - e.g. theta-gamma coupling that increases following a memory encoding cue.

Each PhaseToAmpCoupling may carry a tuple of EventModulation entries:

```
lift_e(t) = (peak_chi_e - chi_baseline) * envelope_e(t)
max_lift(t) = max over all e of lift_e(t)
chi(t) = clip(chi_baseline + max_lift(t), 0, 1)
```

envelope_e(t) is a Tukey (flat-topped raised-cosine) window:

- starts at latency_sec after each event onset
- total width = window_sec
- cosine ramps occupy edge_fraction of the total width on each side
- flat top is 1.0 (full lift to peak_chi)

- multiple onsets combined by elementwise maximum

Parameter	Type	Default	Description
event_label	str	(required)	Name of the event class that triggers this chi lift. Must match a class in the session's event catalog.
peak_chi	float [0,1]	(required)	The chi value reached at the peak of the window (flat-top = 1.0 on the envelope * lift).
window_sec	float > 0	(required)	Total width of the Tukey window in seconds.
latency_sec	float >= 0	0.0	Delay from event onset to start of window in seconds.
edge_fraction	float [0,0.5]	0.25	Fraction of window_sec used for each raised-cosine ramp. 0 = rectangular, 0.5 = Hann window.

Note: If peak_chi < chi_baseline, the modulation produces a negative lift (chi dip). Multiple EventModulation entries on the same coupling are combined by max - the largest lift at each time point wins.

6.3 Phase-to-Phase Coupling (PPC)

PhaseToPhaseCoupling specifies a phase-locking relationship between two oscillations (n:m coupling). In the current implementation PPC edges are used for topological ordering only - no synthesis action is applied. Full PPC synthesis is planned for a future version.

Parameter	Type	Default	Description
driver	str	(required)	Phase driver population ID.
target	str	(required)	Phase target population ID.
coupling_strength	float [0,1]	0.3	Reserved for future synthesis.
delay_ms	float >= 0	15.0	Phase lag in milliseconds. Reserved for future synthesis.
n_to_m_ratio	tuple[int,int]	(1, 1)	n:m frequency ratio (e.g. (4,1) for 4:1 theta:delta). Reserved.

7. Population-to-Channel Projection

The projection step maps each oscillator population to the physical electrode channels that record it. This is controlled by ProjectionConfig.

```
X shape: (n_populations, n_samples) - stacked carrier waveforms
M shape: (n_populations, n_channels) - projection weight matrix
channel_signals = M.T @ X - (n_channels, n_samples)
```

Projection modes

region_match (default)

Entry $M[p, c] = 1.0$ if population p 's region matches channel c 's region, else 0.0. This means a hippocampal oscillation only reaches hippocampal electrodes, and an amygdala oscillation only reaches amygdala electrodes. It is anatomy-aware but binary.

all_identical

Every entry $M[p, c] = 1.0$. All populations reach all channels identically. Useful for single-region simulations or when spatial separation is not needed.

Parameter	Type	Default	Description
mode	str	'region_match'	'region_match' or 'all_identical'. Controls how oscillator populations map to channels.

channel_noise_sd	float ≥ 0	3.0	Standard deviation of per-channel independent Gaussian noise (μV). Added after projection.
------------------	----------------	-----	---

Note: Background (1/f) and line noise are NOT projected through M. They bypass projection and are broadcast identically to all channels.

8. Per-Channel Independent Noise

After projection, background addition, and line noise addition, each channel receives independent Gaussian noise from its own deterministic seed:

```
channel_seed = derive(session_seed, 'channel', channel_index)
rng = np.random.default_rng(channel_seed)
channel_data[j] = channel_signals[j] + rng.normal(0, channel_noise_sd, n_samples)
```

This models electrode-level measurement noise, thermal noise, and impedance differences. Set `channel_noise_sd=0` to disable per-channel noise entirely (useful for isolated testing of coupling properties).

9. Ground-Truth Bundle

Every synthetic session carries a `ground_truth` dict with full provenance. This is what separates PAC Framework from generic signal simulators - you always know the exact true $\chi(t)$ trajectory, phases, and waveforms.

Parameter	Type	Default	Description
pre_coupling_carriers	dict[str, ndarray]		Carrier waveform for each oscillator population, BEFORE any coupling was applied. Shape (n_samples,).
phases	dict[str, ndarray]		Instantaneous phase (wrapped to $[-\pi, \pi]$) for each oscillator population. Shape (n_samples,).
chi_trajectories	dict[str, ndarray]		True $\chi(t)$ time series for each PAC coupling, keyed as 'driver__to__target'. Constant if no EventModulation, time-varying if modulated. Shape (n_samples,).
line_noise	dict[str, ndarray]		Line noise trace for each LineNoisePopulation. Shape (n_samples,).
backgrounds	dict[str, ndarray]		1/f background trace for each BackgroundPopulation. Shape (n_samples,).
projection_matrix	ndarray		The (n_populations, n_channels) projection weight matrix M used to map carriers to channels.
oscillator_order	list[str]		Ordered list of oscillator population IDs matching the row order of the projection matrix and pre_coupling_carriers.
channel_order	list[str]		Ordered list of channel names matching the column order of the projection matrix.
signal_config_json	str		Full JSON serialisation of the SignalConfig used for this session. Enables exact replay.

Example access:

```
session = sessions[0]
gt = session.ground_truth

# True chi trajectory for theta->gamma coupling
chi_t = gt['chi_trajectories']['theta__to__gamma'] # shape (n_samples,)

# Pre-coupling theta carrier
theta_pre = gt['pre_coupling_carriers']['theta'] # shape (n_samples,)

# Theta instantaneous phase
theta_phase = gt['phases']['theta'] # shape (n_samples,)

# Which channels received theta?
M = gt['projection_matrix'] # (n_pops, n_channels)
theta_idx = gt['oscillator_order'].index('theta')
theta_channels = [gt['channel_order'][c] for c, w in enumerate(M[theta_idx]) if w > 0]
```

10. Experiment Design - Sessions & Events

Before synthesising signals you build a skeleton of sessions with scheduled events using `build_sessions()`. This separates experimental design (what happened and when) from signal physics (what the neural signal looks like).

SessionSpec

Parameter	Type	Default	Description
date	str	(required)	ISO date string for the recording (e.g. '2026-01-15').
duration_sec	float > 0	(required)	Session duration in seconds.
task	str	'synthetic'	Free-form task label stored in <code>Session.task</code> .
event_catalog	tuple[EventClasses,]	()	All event classes for this session. Each class is scheduled independently.

EventClass

Defines one class of experimental events. Onsets are sampled from a Poisson process and then filtered by a minimum inter-event gap:

Parameter	Type	Default	Description
name	str	(required)	Event class label (e.g. 'cue_onset', 'response').
rate_hz	float > 0	(required)	Expected events per second (Poisson rate).
min_gap_sec	float >= 0	0.0	Minimum inter-event interval in seconds. Greedy forward walk drops events too close together.

Each `EventClass` gets its own deterministic seed derived from the session seed. Multiple event classes are scheduled independently and then merged and sorted by time into a single `Events` object.

Event scheduling algorithm

```
n = Poisson(rate_hz * duration_sec)
candidates = sorted(Uniform(0, duration_sec), size=n)
accepted = greedy forward walk: accept t if t - last_accepted >= min_gap_sec
```

Note: The Poisson count draw means the actual number of events is random around the expected count. Use `rate_hz=0.5` to get roughly 1 event per 2 seconds on average in a 60-second session (expected ~30 events, Poisson-distributed).

11. Using the GUI - Tab by Tab

Launch the GUI by running: `python -m pac_framework` (or the desktop shortcut). The window has three tabs and four toolbar buttons. The workflow is: configure in Tab 1 -> press Build -> configure in Tab 2 -> press Generate -> inspect in Tab 3 -> Save.

Toolbar buttons

Parameter	Type	Default	Description
Build Timeline & Events (Ctrl+B)			Validates all config, calls <code>build_sessions()</code> , creates zero-filled channels with scheduled events. Must be done before Generate.
Generate Signals (Ctrl+G)			Calls <code>generate_signals()</code> on the current state. Switches to the Data Browser tab when done.
Save (Ctrl+S)			Saves the subject to <code>subjects/<name>/</code> as HDF5 session files + manifest.json.
Load (Ctrl+L)			Opens a folder picker to load a previously saved subject directory.

11.1 Subject Designer Tab

This tab defines WHO the subject is and WHAT happened in each session. It does not define the neural signals - that is Tab 2.

Subject Metadata group

- Subject Name: arbitrary string identifier stored in `Session.subject_id`.

- Master

Sessions group

Each row defines one recording session. Right-click or use the Add/Remove buttons. Columns: Date (ISO format), Duration (seconds), Task (label string). The 'Edit Catalog' button opens the per-session event catalog editor.

Event Catalog editor

Each session has its own event catalog. Default catalog contains three classes: `cue_onset`, `decision_made`, `outcome` - each at 0.2 Hz with 3s minimum gap. Edit columns: Class Name, Rate (Hz), Min Gap (s). Class names must be unique within a session. Any class name you define here becomes available as an event trigger in the Coupling Modulation dialog (Section 6.2).

Channel Layout group

Defines the electrode geometry. Each row is one shaft (electrode array): Shaft (name, e.g. 'LA'), Region (e.g. 'amygdala'), Contacts (integer count), Spacing (mm between adjacent contacts). Channel names are auto-generated as `ShaftName_ContactIndex` (e.g. LA_00, LA_01, ..., LA_07). The region label must match the region string of oscillator populations for `region_match` projection to work.

11.2 Signal Config Tab

This tab defines the neural signal model for the currently selected session. The session selector at the top lets you switch between sessions; changes are stored per-session. The 'Apply to All Sessions' button copies the current config to every session.

Oscillators group

Each row is one OscillatorPopulation. Required columns: ID (unique), Center Freq (Hz), Bandwidth (Hz), Amplitude (uV), Region. The 'Advanced' button opens a dialog for WaveformShape, PAFDrift, BurstConfig, and ArtifactConfig parameters.

Advanced Oscillator Dialog

Grouped into four sections:

- Waveform Shape: Peak/trough sharpness [-1,1] and Rise/decay asymmetry [-1,1].

- PAF Drift

Backgrounds (1/f) group

Each row is one BackgroundPopulation. Columns: ID, Slope (exponent), Knee (Hz), Amplitude (uV).

Line Noise group

Each row is one LineNoisePopulation. Columns: ID, Frequency (Hz), Harmonics (comma-separated integers), Amplitudes (comma-separated uV values). Example: harmonics='1,2,3', amplitudes='1.0,0.3,0.1' gives 60+120+180 Hz at those amplitudes.

Couplings group - PAC table

Each row defines one PhaseToAmpCoupling. Columns: Driver (oscillator ID dropdown), Target (oscillator ID dropdown), chi (baseline coupling depth), phi_0 in degrees, kappa. The 'Modulation...' button opens the Event Modulation dialog.

Event Modulation Dialog

Accessible via the 'Modulation...' button on each PAC coupling row. Each row in this dialog is one EventModulation entry. Columns: Event label (dropdown of the current session's event catalog), Peak chi [0,1], Window (s), Latency (s), Edge fraction [0,0.5]. Add/Remove rows with the buttons. Multiple rows combine by max at synthesis time.

Couplings group - PPC table

Each row defines one PhaseToPhaseCoupling. Columns: Driver, Target, Strength, Delay (ms), N:M ratio. PPC currently imposes ordering constraints only - no synthesis action.

Projection group

Mode dropdown: 'region_match' (default) or 'all_identical'. Channel Noise SD: Gaussian noise standard deviation in uV added independently to each channel after projection.

11.3 Data Browser Tab

Displays the generated waveforms. Shows one session at a time. All channels are displayed in a scrollable multi-trace viewer. Before Generate Signals is pressed, channels show flat zero. The timeline preview at the bottom shows the event raster (one row per event class, dots at onset times).

Note: The Data Browser view is refreshed automatically after Generate Signals and after Load. Editing any parameter in Tab 1 or Tab 2 after generating signals will zero out the channel data with the message 'Signals invalidated'.

12. Programmatic API (no GUI)

The entire synthesis pipeline can be used from Python without launching the GUI. Import `pac_framework` as `pac` and use the public functions directly.

```
import pac_framework as pac

# 1. Define electrode layout
channel_info = pac.channel_info_from_shafts([
    {'shaft': 'LA', 'region': 'amygdala', 'contacts': 8, 'spacing_mm': 1.5},
    {'shaft': 'LH', 'region': 'hippocampus', 'contacts': 8, 'spacing_mm': 1.5},
])

# 2. Define sessions
specs = [
    pac.SessionSpec(
        date='2026-01-01', duration_sec=120.0, task='memory_task',
        event_catalog=(
            pac.EventClass(name='cue_onset', rate_hz=0.5, min_gap_sec=2.0),
            pac.EventClass(name='response', rate_hz=0.4, min_gap_sec=1.5),
        ),
    ),
]

# 3. Build skeleton sessions (zero channels + events)
sessions = pac.build_sessions(
    subject_name='sub-01', seed=42, sfreq=1000.0,
    session_specs=specs, channel_info=channel_info,
)

# 4. Configure signals
theta = pac.OscillatorPopulation(
    id='theta', center_frequency=6.0, bandwidth=2.0,
    amplitude=50.0, region='hippocampus',
    burst=pac.BurstConfig(mode='continuous'),
    paf_drift=pac.PAFDrift(sigma_hz=0.5, tau_seconds=8.0),
)
gamma = pac.OscillatorPopulation(
    id='gamma', center_frequency=70.0, bandwidth=10.0,
    amplitude=10.0, region='hippocampus',
    burst=pac.BurstConfig(mode='bursty', rate_hz=1.5),
)
config = pac.SignalConfig(
    populations=[theta, gamma],
    couplings=[
        pac.PhaseToAmpCoupling(
            driver='theta', target='gamma',
            chi=0.3, phi_0=0.0, kappa=3.0,
            event_modulations=(
                pac.EventModulation(
                    event_label='cue_onset', peak_chi=0.85,
                    window_sec=1.5, latency_sec=0.1, edge_fraction=0.25,
                ),
            ),
        ),
    ],
],
```

```
        projection=pac.ProjectionConfig(mode='region_match', channel_noise_sd=3.0),
    )

# 5. Synthesise
sessions = pac.generate_signals(
    sessions=sessions, signal_configs=[config],
    master_seed=42, subject_name='sub-01',
)

# 6. Access data
raw    = sessions[0].channels.data           # (n_channels, n_samples)
times  = sessions[0].timeline.times()        # (n_samples,) in seconds
chi_t  = sessions[0].ground_truth['chi_trajectories']['theta__to__gamma']
hpc    = sessions[0].channels.get_channels(region='hippocampus')
```

13. Saving & Loading Subjects

The Save action writes to `subjects/<subject_name>/` with one HDF5 file per session plus a JSON manifest. This structure can be reloaded by the GUI or by code.

On-disk structure

```
subjects/
  sub-01/
    manifest.json      Schema version + gui_config + supplementary config
    session_000.h5     HDF5 for session 0 (channels, events, ground truth)
    session_001.h5     HDF5 for session 1
    ...
```

HDF5 session file layout

```
session_000.h5
  /identity            subject_id, session_id, task, date_recorded, origin
  /timeline            sfreq, n_samples, tmin
  /channels/data       (n_channels, n_samples) float64, gzip compressed
  /channels/info/      per-column datasets: name, type, shaft, region, ...
  /events/samples      int64 onset indices
  /events/labels       UTF-8 encoded strings
  /events/codes        int64 numeric codes
  /ground_truth/pre_coupling_carriers/<pop_id> float64
  /ground_truth/phases/<pop_id> float64
  /ground_truth/chi_trajectories/<coupling_key> float64
  /ground_truth/backgrounds/<pop_id> float64
  /ground_truth/line_noise/<pop_id> float64
  /ground_truth/projection_matrix (n_pop, n_ch) float64
```

Schema version migration

The manifest carries a `schema_version` field. When loading an older file the migration chain automatically updates it to the current version (0.10.0). The following migrations are applied in sequence:

Parameter	Type	Default	Description
0.0.0 -> 0.1.0			Add <code>channel_layout</code> to <code>gui_config</code> .
0.1.0 -> 0.2.0			Add <code>event_catalog</code> to <code>gui_config</code> .
0.2.0 -> 0.3.0			Add <code>trial_structure</code> to <code>gui_config</code> .
0.3.0 -> 0.4.0			Restructure flat <code>gui_config</code> keys into nested sub-dicts; add subject metadata.
0.4.0 -> 0.5.0			Expand uniform sessions spec into per-session row list; move <code>sfreq</code> to top level.
0.5.0 -> 0.6.0			Move subject-level <code>event_catalog</code> into each session row.
0.6.0 -> 0.7.0			Add <code>signals_populated</code> flag and default <code>signal_config</code> .
0.7.0 -> 0.8.0			Replace single <code>signal_config</code> with per-session list <code>session_signal_configs</code> .

0.8.0 -> 0.9.0





Add ArtifactConfig defaults to every oscillator population.

0.9.0 -> 0.10.0			Add event_modulations: [] to every PAC coupling entry.
-----------------	--	--	--

14. Deterministic Reproducibility

All randomness in the framework is derived from a single master seed. The seed derivation uses BLAKE2b hashing to produce non-overlapping, uncorrelated sub-seeds for every stochastic component.

```
derive(parent_seed: int, *labels: str | int) -> int
```

Encodes: `struct.pack('>q', parent_seed) + ':'.join(str(l) for l in labels).encode()`

Returns: `int.from_bytes(blake2b(encoded, digest_size=8).digest(), 'big')`

The seed tree looks like:

```
master_seed (42)
  derive('subject', 'sub-01') -> subject_seed
    derive('session', 0)      -> session_0_seed
      derive('population', 'theta') -> theta_seed
        derive('paf_drift')      -> paf_seed      (theta OU drift)
        derive('burst')         -> burst_seed     (theta burst envelope)
        derive('artifact')      -> artifact_seed  (theta artifacts)
      derive('population', 'gamma') -> gamma_seed
      derive('channel', 0)       -> channel_0_seed (per-channel noise)
      derive('channel', 1)       -> channel_1_seed
      derive('events', 'cue_onset') -> event_seed  (event scheduling)
    derive('session', 1)      -> session_1_seed
    ...
```

Changing the master seed changes ALL waveforms. Changing a population's id changes only that population's branch. Adding a new session changes only that session's branch (existing sessions unaffected). This means you can create multiple independent realisations of the same experiment by just incrementing the master seed.

15. Parameter Quick Reference

SignalConfig

Parameter	Type	Default	Description
populations	list[Population]	[]	List of OscillatorPopulation, BackgroundPopulation, LineNoisePopulation.
couplings	list[Coupling]	[]	List of PhaseToAmpCoupling, PhaseToPhaseCoupling.
projection	ProjectionConfig	default	Projection mode and per-channel noise level.

OscillatorPopulation - full parameter list

Parameter	Type	Default	Description
id	str	required	Unique population identifier.
center_frequency	float > 0	required	Center frequency in Hz.
bandwidth	float >= 0	required	Bandwidth in Hz. 0 = pure sine.
amplitude	float > 0	required	Output amplitude in uV (sets carrier std).
region	str	required	Brain region, must match channel layout for region_match projection.
waveform_shape.peak_trough_sharpness	float [-1,1]	0.0	Harmonic sharpness.
waveform_shape.rise_decay_asymmetry	float [-1,1]	0.0	Harmonic asymmetry.
paf_drift.sigma_hz	float >= 0	0.0	OU frequency drift magnitude.
paf_drift.tau_seconds	float > 0	5.0	OU frequency drift correlation time.
burst.mode	str	'continuous'	'continuous' or 'bursty'.
burst.rate_hz	float > 0	0.5	Burst rate in Hz.
burst.duration_cycles_mean	float > 0	3.0	Mean burst duration in carrier cycles.
burst.duration_cycles_sd	float >= 0	0.5	Burst duration SD in carrier cycles.
burst.refractory_cycles	float >= 0	1.0	Refractory period in carrier cycles.
artifact.rate_hz	float >= 0	0.0	Artifact injection rate. 0 = disabled.
artifact.amplitude_mult	float > 0	3.0	Artifact amplitude as carrier std multiple.
artifact.width_samples	int > 0	5	Artifact transient width in samples.

PhaseToAmpCoupling - full parameter list

Parameter	Type	Default	Description
driver	str	required	Driver population ID (phase source).
target	str	required	Target population ID (amplitude modulated).
chi	float [0,1]	0.5	Baseline coupling depth.
phi_0	float (rad)	0.0	Preferred phase in radians (GUI uses degrees).
kappa	float >= 0	2.0	Von Mises concentration (coupling sharpness).
event_modulations[*].event_label	str	required	Session event class that triggers this lift.
event_modulations[*].peak_chi	float [0,1]	required	Chi value reached at window peak.
event_modulations[*].window_sec	float > 0	required	Tukey window total duration in seconds.
event_modulations[*].latency_sec	float >= 0	0.0	Delay from event onset to window start.
event_modulations[*].edge_fraction	float [0,0.5]	0.25	Cosine ramp fraction (0=rectangular, 0.5=Hann).

BackgroundPopulation - full parameter list

Parameter	Type	Default	Description
id	str	required	Unique identifier.
slope	float > 0	1.5	1/f spectral exponent.
knee_hz	float >= 0	0.0	Spectral knee in Hz. 0 = pure power law.
amplitude	float > 0	1.0	Output amplitude in uV.

LineNoisePopulation - full parameter list

Parameter	Type	Default	Description
id	str	required	Unique identifier.
frequency	float > 0	60.0	Fundamental frequency in Hz.
harmonics	list[int]	[1, 2, 3]	Harmonic indices (1 = fundamental).
amplitude_per_harmonic	list[float]	[1.0,0.3,0.1]	RMS amplitude per harmonic in uV.